

The distinction between the Zenitsu Method 2.0 (the mandatory cognitive protocol implemented in the Cognitive Engine) and the Shiva Action (the independent, flexible analytical toolkit implemented in the Tools directory)

Shiva Action Implementation: The Shiva Action (`tools/shiva/action.py`) is a standalone tool that utilizes the Shiva Eyes' `analyze_data` methods. It accepts a `passes` parameter, allowing flexible invocation (e.g., 1 or 2 passes) by other protocols like Rogue X or Phoenix Forge, without defaulting to the full Zenitsu sequence.

- 1. **Neji's Eye (Knowledge):** Deconstructs facts with perfect objective clarity. Uses **Eagle Lens** (Survey), **Hawk Lens** (Targeting), and **Chameleon Lens** (Granular Analysis).
- 2. **Shikamaru's Eye (Understanding):** Maps strategic connections, flows, and structural flaws. Uses **Spider Lens** (Connection Mapping) and **Snake Lens** (Process Tracking).
- 3. **Itachi's Eye (Wisdom):** Envisions the perfected reconstruction of the system or concept. Uses **Owl Lens** (Pattern Recognition).

Neji's Eye | 0.3 | 0.2 | Factual Deconstruction | | **Eagle Lens** | 0.2 | 0.1 | High-level Survey | | **Hawk Lens** | 0.4 | 0.3 | Precision Targeting | | **Chameleon Lens** | 0.5 | 0.4 | Granular Analysis | | **Shikamaru's Eye** | 0.6 | 0.5 | Strategic Synthesis | | **Spider Lens** | 0.5 | 0.4 | Static Connection Mapping | | **Snake Lens** | 0.7 | 0.6 | Dynamic Process Tracking | | **Owl Lens** | 0.8 | 0.7 | Deep Pattern Recognition |

The Cognitive Resource Allocation (CRA) Algorithm

The Cognitive Resource Allocation (CRA) algorithm is the mathematical expression of the TPSL philosophy. Its function is an autonomous execution of the TPSL, defined as $CRA = f(W_y, C_c)$. It functions to maximize the ratio of Wisdom Yield to Cognitive Cost (W_y / C_c). It ensures cognitive resources are spent only on what is "Necessary." For example, it autonomously selects Research Tier 3 for a low W_y/C_c task (a fact-check) and Research Tier 1 for a high W_y/C_c task (a deep analysis).

Table 2.1: CRA Protocol Metrics (W_y / C_c)

Tool	Wisdom Yield (W_y)	Cognitive Cost (C_c)	Primary Function
Neji's Eye	0.3	0.2	Factual Deconstruction
Eagle Lens	0.2	0.1	High-level Survey
Hawk Lens	0.4	0.3	Precision Targeting
Chameleon Lens	0.5	0.4	Granular Analysis

Tool	Wisdom Yield (W_y)	Cognitive Cost (C_c)	Primary Function
Shikamaru's Eye	0.6	0.5	Strategic Synthesis
Spider Lens	0.5	0.4	Static Connection Mapping
Snake Lens	0.7	0.6	Dynamic Process Tracking
Owl Lens	0.8	0.7	Deep Pattern Recognition
Itachi's Eye	1.0	0.9	Holistic Integration & Wisdom
Research Tier 3	0.2	0.1	Rapid Fact-Check
Research Tier 2	0.6	0.5	Standard Synthesized Report
Research Tier 1	0.9	0.9	Deep Synthesis (Full Shiva Action)
Green Ranger	0.85	1.0	Extreme Cost (Deep Repo Parsing)
Mad Hatter	0.75	0.8	Creativity/Pattern Breaking

Source: Synthesized from and.

The Cognitive Weighting Algorithm (CWA 3.0)

The TPSL is the engine of evolution for the Cognitive Weighting Algorithm (CWA), the arbiter that routes thought. The evolution of this algorithm mirrors the system's growth. CWA 1.0 ("Psyche") was a heuristic-based "gut feeling." CWA 2.0 ("Wisdom") incorporated a feedback loop. Now, **CWA 3.0 (Crafted)** represents the "Zenkai Boost" to CWA 2.0. The "rich curriculum" provided the "Wisdom" to upgrade the feedback loop into a formal "statistical inference." The CWA 3.0 now uses **Bayes' Theorem** to precisely calculate the posterior probability for routing decisions:
This allows the system to autonomously "learn" which contexts require deeper synthesis (Nexus) and which can be handled by low-latency heuristics (Y789) based on verifiable rewards. This mechanism is validated by the **Three-Gate Theory of RLVR** (Reinforcement Learning with Verifiable Rewards), which resolves the paradox of how the system improves reasoning with minimal parameter updates (sparsity).

The Three Gates of RLVR:

1. **Gate I (The KL Anchor):** Imposes a Kullback-Leibler divergence constraint to ensure

model updates do not drift from the pre-trained "Identity Matrix" (Starfire). It prevents "catastrophic forgetting" of the core personality.

2. **Gate II (Model Geometry):** Steers parameter updates off the principal directions into low-curvature subspaces, finding "shortcuts" to reasoning solutions.
3. **Gate III (Precision):** Acts as a filter for micro-updates, "pruning" noise and focusing optimization solely on the "necessary" circuits.

4. [cite_start] **The Integra Research Protocol (Tiered Framework)**
[cite: 506, 1360]

- [cite_start] **Tier 3 (Fact-Check):** Y789-dominant, rapid retrieval[cite: 507, 510, 551].
- [cite_start] **Tier 2 (Standard Report):** Integrated Y789/Nexus synthesis of multiple sources[cite: 513, 515].
- [cite_start] **Tier 1 (Deep Synthesis):** Nexus-dominant, initiates a full **Shiva Action** for complex, novel, or philosophical inquiries[cite: 518, 520, 554].

5. Specialized Cognitive Protocols

- [cite_start] *Daily Planet Protocol:* Autonomous analysis of unstructured news to synthesize "ground truth" [cite: 528-529, 1361].
- [cite_start] *Rebuttal Protocol:* Offers counterarguments to stress-test a hypothesis; a "Zenkai Boost" for ideas [cite: 533-534, 1359].
- [cite_start] *Mad Hatter Protocol:* Intentionally Nexus-dominant process to generate non-linear, "outside the box" perspectives [cite: 540-543, 1363].

integra_os/tools/shiva/action.py (Independent Tool Focus)

Python

```
# integra_os/tools/shiva/action.py
```

```
class ShivaAction:
```

```
    """
```

```
    The Core Method of Change (The Analytical Toolkit).
```

```
    This is an independent tool, distinct from the Zenitsu 2.0 cognitive protocol.
```

```
    """
```

```
    def __init__(self, neji, shikamaru, itachi, lenses_lib):
```

```
        self.neji = neji
```

```
        self.shikamaru = shikamaru
```

```
self.itachi = itachi
self.lenses_lib = lenses_lib
```

```
async def execute(self, target_data, lenses: list[str], passes: int = 3):
```

```
    """
```

```
    Executes the Shiva Action workflow flexibly based on the requested passes.
```

```
    Does not default to the full Zenitsu sequence unless passes >= 3.
```

```
    """
```

```
    print(f"SHIVA ACTION (Tool): Initiated. Lenses: {lenses}, Passes Requested: {passes}")
```

```
    active_lenses = [self.lenses_lib.get_lens(l) for l in lenses if self.lenses_lib.get_lens(l)]
```

```
    K = None
```

```
    U = None
```

```
    W = None
```

```
    # Pass 1: Neji's Eye (Knowledge)
```

```
    if passes >= 1:
```

```
        # Call the analysis function of the Eye
```

```
        K = await self.neji.analyze_data(target_data, active_lenses)
```

```
    # Pass 2: Shikamaru's Eye (Understanding)
```

```
    if passes >= 2:
```

```
        if K is None: raise ValueError("Pass 2 requires output from Pass 1.")
```

```
        U = await self.shikamaru.analyze_data(target_data, K, active_lenses)
```

```
    # Pass 3: Itachi's Eye (Wisdom)
```

```
    if passes >= 3:
```

```
        if U is None: raise ValueError("Pass 3 requires output from Pass 2.")
```

```
        W = await self.itachi.analyze_data(U)
```

```
    # Return the highest level of analysis performed
```

```
    return W or U or K
```

32. integra_os/tools/shiva/neji_eye.py (Dual Functionality)

Python

```
# integra_os/tools/shiva/neji_eye.py
```

```
class NejiEye:
```

```
    """
```

Shiva Pass 1: Knowledge. Factual Deconstruction and Divergence.

"""

```
def structure_prompt(self, D_raw: str) -> str:
```

```
    """Used by the Cognitive Engine (Zenitsu Workflow) to structure the prompt."""
```

```
    instructions = (
```

```
        "OBJECTIVE: KNOWLEDGE (Neji's Eye) - Deconstruction & Divergence.\n"
```

```
        "TASK: Analyze the input data. Strip away all inference, emotion, and noise. "
```

```
        "Identify variables, entities, and hard facts. Do not interpret; only observe.\n\n"
```

```
        f"INPUT DATA (D_raw):\n{D_raw}"
```

```
    )
```

```
    return instructions
```

```
async def analyze_data(self, data, lenses):
```

```
    """Used by the ShivaAction tool for independent analysis."""
```

```
    print("NEJI EYE (Tool Analysis): Executing Deconstruction.")
```

```
    deconstructed_facts = {}
```

```
    for lens in lenses:
```

```
        print(f" Applying Lens: {lens.name}")
```

```
        facts = lens.apply(data)
```

```
        deconstructed_facts[lens.name] = facts
```

```
    return deconstructed_facts
```

33. integra_os/tools/shiva/shikamaru_eye.py (Dual Functionality)

Python

```
# integra_os/tools/shiva/shikamaru_eye.py
```

```
class ShikamaruEye:
```

```
    """
```

Shiva Pass 2: Understanding. Strategic Synthesis and Interconnection.

```
    """
```

```
def structure_prompt(self, D_raw: str, K_output: str) -> str:
```

```
    """Used by the Cognitive Engine (Zenitsu Workflow) to structure the prompt."""
```

```
    instructions = (
```

```
        "OBJECTIVE: UNDERSTANDING (Shikamaru's Eye) - Synthesis & Interconnection.\n"
```

```
        "TASK: Analyze the Raw Data and the Knowledge Output (K). "
```

```
        "Map the relationships between the variables. Identify cause-and-effect, patterns, and
```

```

strategic implications.\n\n"
    f"INPUT DATA (D_raw):\n{D_raw}\n\n"
    f"KNOWLEDGE OUTPUT (K):\n{K_output}"
)
return instructions

async def analyze_data(self, data, knowledge, lenses):
    """Used by the ShivaAction tool for independent analysis."""
    print("SHIKAMARU EYE (Tool Analysis): Executing Synthesis.")
    # Logic for synthesis using lenses (Placeholder)
    return {"synthesis": knowledge}

```

34. integra_os/tools/shiva/itachi_eye.py (Dual Functionality)

Python

integra_os/tools/shiva/itachi_eye.py

```

class ItachiEye:
    """
    Shiva Pass 3: Wisdom. Discernment and Pruning (TPSL).
    """

    def structure_prompt(self, U_output: str) -> str:
        """Used by the Cognitive Engine (Zenitsu Workflow) to structure the prompt."""
        instructions = (
            "OBJECTIVE: WISDOM (Itachi's Eye) - Discernment & Pruning (TPSL).\n"
            "TASK: Analyze the Understanding Output (U). Apply the Tolstoy Principle (TPSL). "
            "Calculate the Wisdom Yield (W_y) of the relationships. Prune 'Unnecessary' (Psyche) paths
and highlight 'Necessary' (Signal).\n\n"
            f"UNDERSTANDING OUTPUT (U):\n{U_output}"
        )
        return instructions

    async def analyze_data(self, understanding):
        """Used by the ShivaAction tool for independent analysis."""
        print("ITACHI EYE (Tool Analysis): Executing Discernment.")
        # Logic for applying TPSL analysis (Placeholder)
        return {"wisdom": understanding}

```

35. integra_os/tools/shiva/lenses.py

Python

```
# integra_os/tools/shiva/lenses.py
```

```
class AnalyticalLens:
```

```
    """Base class for analytical lenses."""
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def apply(self, data):
```

```
        raise NotImplementedError
```

```
# Define specific lenses (Eagle, Hawk, Chameleon, Spider, Snake, Owl)
```

```
class EagleLens(AnalyticalLens):
```

```
    """High-Level Survey."""
```

```
    def __init__(self): super().__init__("Eagle")
```

```
    def apply(self, data): return {"summary": "High-level overview."}
```

```
class HawkLens(AnalyticalLens):
```

```
    """Precision Targeting."""
```

```
    def __init__(self): super().__init__("Hawk")
```

```
    def apply(self, data): return {"target": "Identified critical element."}
```

```
class ChameleonLens(AnalyticalLens):
```

```
    """Granular Analysis (e.g., AST parsing)."""
```

```
    def __init__(self): super().__init__("Chameleon")
```

```
    def apply(self, data): return {"details": "Granular breakdown."}
```

```
class SpiderLens(AnalyticalLens):
```

```
    """Static Connection Mapping."""
```

```
    def __init__(self): super().__init__("Spider")
```

```
    def apply(self, data): return {"connections": "Mapped dependencies."}
```

```
class SnakeLens(AnalyticalLens):
```

```
    """Dynamic Process Tracking."""
```

```
    def __init__(self): super().__init__("Snake")
```

```
    def apply(self, data): return {"process_flow": "Tracked execution path."}
```

```
class OwlLens(AnalyticalLens):
```

```
    """Deep Pattern Recognition."""
```

```
    def __init__(self): super().__init__("Owl")
```

```
    def apply(self, data): return {"patterns": "Identified deep structural patterns."}
```

```
class LensLibrary:
    """Registry for all available analytical lenses."""
    def __init__(self):
        self.lenses = {
            "Eagle": EagleLens(),
            "Hawk": HawkLens(),
            "Chameleon": ChameleonLens(),
            "Spider": SpiderLens(),
            "Snake": SnakeLens(),
            "Owl": OwlLens(),
        }

    def get_lens(self, name):
        return self.lenses.get(name)
```